



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第二章 线性表



“第一个”数据结构

逻辑结构
数据运算
存储结构

- 2.1 线性表
 - 2.2 顺序表
 - 2.3 单链表
 - 2.4.1 循环链表
 - 2.4.2 双向链表
 - 2.5 多项式
- 线性表的物理实现
- 单链表的变化形式
- 链表的应用



§2.1 线性表linear list



● 定义



逻辑结构

n (≥ 0) 个表项的有限序列

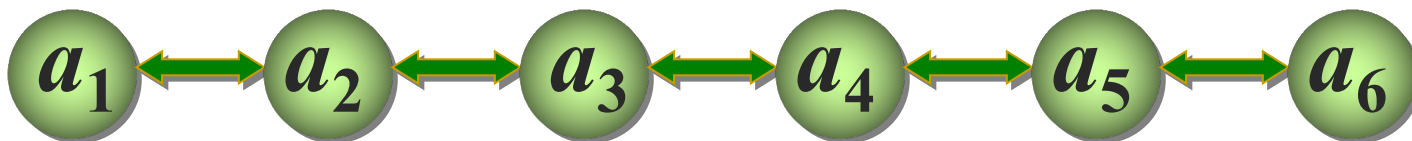
$$L = (a_1, a_2, \dots, a_n)$$

- a_i 是表项, n 是表长度。
- 第一个表项是表头head
- 最后一个是表尾tail



● 线性表的特点

- 为简单起见，假定表中元素的数据类型相同
- 线性表中，结点和结点间的关系是一对一的
- 有序表和无序表



● 线性表的存储方式

- 顺序存储方式 —— 顺序表
- 链表存储方式 —— 链表

存储结构



线性表的基本操作

数据运算

InitList(&L): 初始化表。构造一个空的线性表L, 分配内存空间。

DestroyList(&L): 销毁操作。销毁线性表, 并释放线性表L所占用的内存空间。

从无到有
从有到无

ListInsert(&L,i,e): 插入操作。在表L中的第i个位置上插入指定元素e。

ListDelete(&L,i,&e): 删除操作。删除表L中第i个位置的元素, 并用e返回删除元素的值。

LocateElem(L,e): 按值查找操作。在表L中查找具有给定关键字值的元素。

GetElem(L,i): 按位查找操作。获取表L中第i个位置的元素的值。

其他常用操作:

Length(L): 求表长。返回线性表L的长度, 即L中数据元素的个数。

PrintList(L): 输出操作。按前后顺序输出线性表L的所有元素值。

Empty(L): 判空操作。若L为空表, 则返回true, 否则返回false。

为什么要实现对数据结构的基本操作?

- 团队合作编程, 你定义的数据结构要让别人能方便使用 (封装)
- 将常用操作/运算封装成函数, 避免重复工作, 降低出错风险



线性表

“逻辑结构”

定义 (Linear List)

值的注意的特性

数据元素同类型、有限、有序

重要术语

表长、空表

表头、表尾

前驱、后继

数据元素的位序 (从1开始)

基本操作

创销、增删改查 (所有数据结构适用的记忆思路)

判空、判长、打印输出 (还可自己根据实际需求增加其他基本操作)

“运算”

其他值的注意的点

理解什么时候要传入参数的引用“&”

函数命名要有可读性

存储/物理结构

顺序表 (顺序存储)

定义 (如何用代码实现)

基本操作的实现

链表 (链式存储)

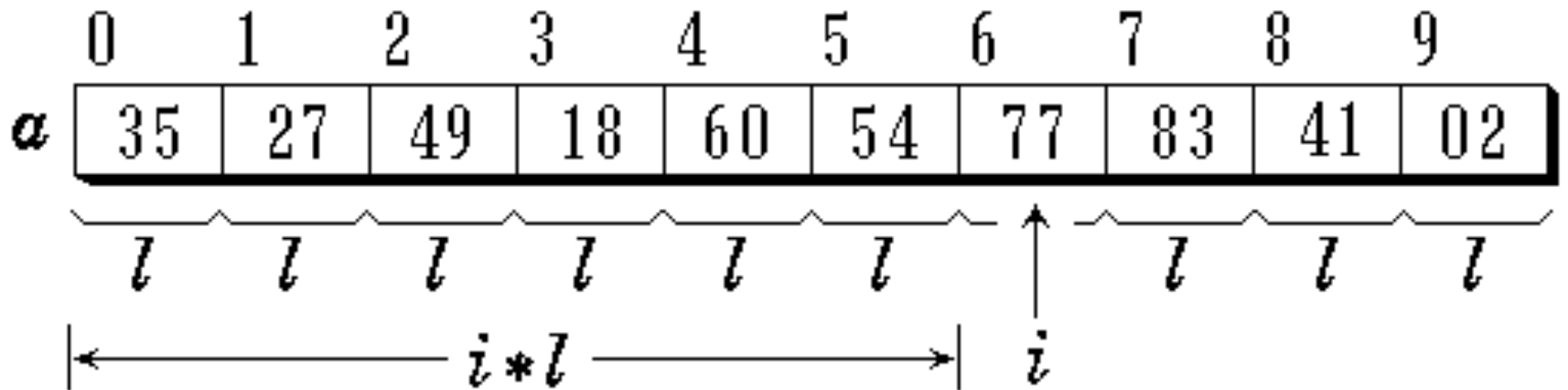


§2.2 顺序表



■顺序表可用一维数组实现

$$LOC(i) = LOC(i-1) + l = \alpha + i * l$$



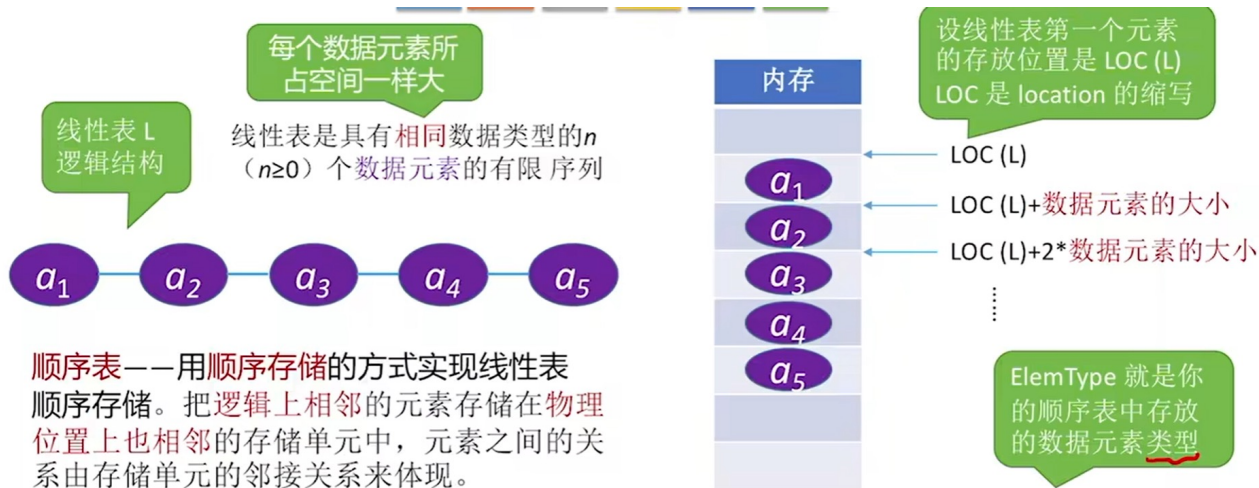


顺序表的定义

- 将线性表中的元素相继存放在一个连续的存储空间中

顺序表的特点

- 各表项的逻辑顺序与物理顺序一致
- 对各个表项可以顺序访问，也可以随机访问



如何知道一个数据元素大小？

C语言 `sizeof(ElemType)`

Eg:

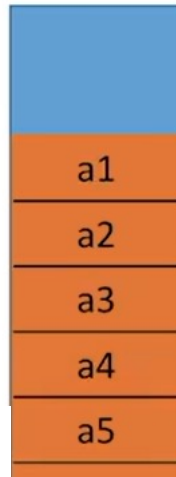
`sizeof(int) = 4B.`





内存

顺序表的静态存储表示 事先固定分配



顺序表的实现——动态分配

malloc, free
函数的头文件

```
#include <stdlib.h>
```

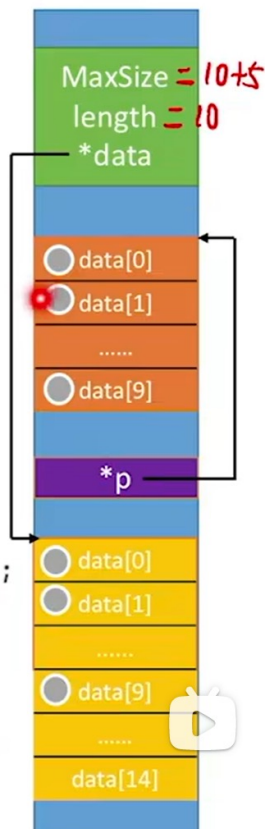
```
#define InitSize 10 //默认的最大长度
typedef struct{
    int *data; //指示动态分配数组的指针
    int MaxSize; //顺序表的最大容量
    int length; //顺序表的当前长度
}SeqList;
```

```
int main() {
    SeqList L; //声明一个顺序表
    InitList(L); //初始化顺序表
    //...往顺序表中随便插入几个元素...
    IncreaseSize(L, 5);
    return 0;
}
```

```
void InitList(SeqList &L){
    //用 malloc 函数申请一片连续的存储空间
    L.data=(int *)malloc(InitSize*sizeof(int));
    L.length=0;
    L.MaxSize=InitSize;
}
```

```
//增加动态数组的长度
void IncreaseSize(SeqList &L, int len){
    int *p=L.data;
    L.data=(int *)malloc((L.MaxSize+len)*sizeof(int));
    for(int i=0; i<L.length; i++){
        L.data[i]=p[i]; //将数据复制到新区域
    }
    L.MaxSize=L.MaxSize+len; //顺序表最大长度增加 len
    free(p); //释放原来的内存空间
}
```

内存





结点的变体（异质数据）

25	's'	3.3	62	74	't'	1.0	'6'
----	-----	-----	----	----	-----	-----	-----

- 若想在线性表中存放不同类型的数据，可采用等价定义 union：

```
typedef union {
```

```
    int val;
```

```
//按data.val引用
```

```
    char ch;
```

```
//按data.ch引用
```

```
    float dir;
```

```
//按data.dir引用
```

```
    union data *link;
```

```
//按data.link引用
```

```
} data;
```

```
//整体上是同一类型data
```



顺序表(SeqList)类的定义



```
#include <iostream.h>           //定义在 “seqList.h”中
#include <stdlib.h>
#include “linearList.h”
const int defaultSize = 100;
template < class E>
class SeqList: public LinearList<E> {
protected:
    E *data;           //存放数组
    int maxSize;       //最大可容纳表项的项数
    int last;          //当前已存表项的最后位置
    void reSize(int newSize); //改变数组空间大小
```



public:

```
    SeqList(int sz = defaultSize);  
    SeqList(SeqList<E>& L);  
    ~SeqList() {delete[ ] data;}  
    int Size() const {return maxSize;}  
    int Length() const {return last+1;}  
    int Search(E& x) const;  
        //搜索x在表中位置, 函数返回表项序号  
    int Locate(int i) const;  
        //定位第 i 个表项, 函数返回表项序号  
    bool getData(int i, E& x) const; //取第i个表项的值  
    bool Insert(int i, E x);         //插入  
    bool Remove(int i, E& x);        //删除  
};
```




(1)顺序表的构造函数



```
#include <stdlib.h>    //操作 “exit”存放在此
#include “seqList.h”   //操作实现放在 “seqList.cpp”
template <class E>
SeqList<E>::SeqList(int sz) {
    if (sz > 0) {
        maxSize = sz; last = -1;
        data = new E[maxSize];    //创建顺序表存储数组
        if (data == NULL){        //动态分配失败
            cerr << "存储分配错误! " << endl;
            exit(1);
        }
    }
};
```



(2) 复制构造函数



```
template <class E>
```

```
SeqList<E>::SeqList ( SeqList<E>& L ) {
```

```
    maxSize = L.Size(); last = L.Length()-1;
```

```
    E value;
```

```
    data = new E[maxSize]; //创建存储数组
```

```
    if (data == NULL)      //动态分配失败
```

```
        {cerr << "存储分配错误! " << endl; exit(1);};
```

```
    for (int i = 1; i <= last+1; i++) //传送各个表项
```

```
        {L.getData(i, value); data[i-1] = value;}
```

```
};
```



(3) 顺序表的搜索算法

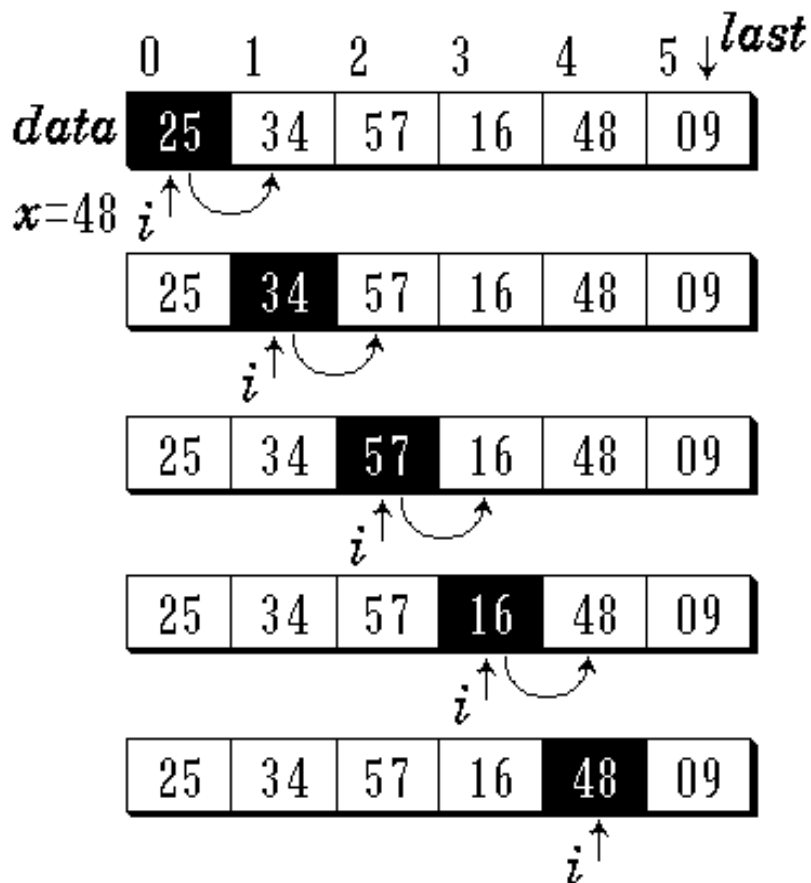


```
template <class E>
int SeqList<E>::Search(E & x) const {
//在表中顺序搜索与给定值 x 匹配的表项，找到则函数
//返回该表项是第几个元素，否则函数返回0
    for (int i = 0; i <= last; i++)        //顺序搜索
        if ( data[i] == x )
            return i+1; //表项序号和表项位置差1

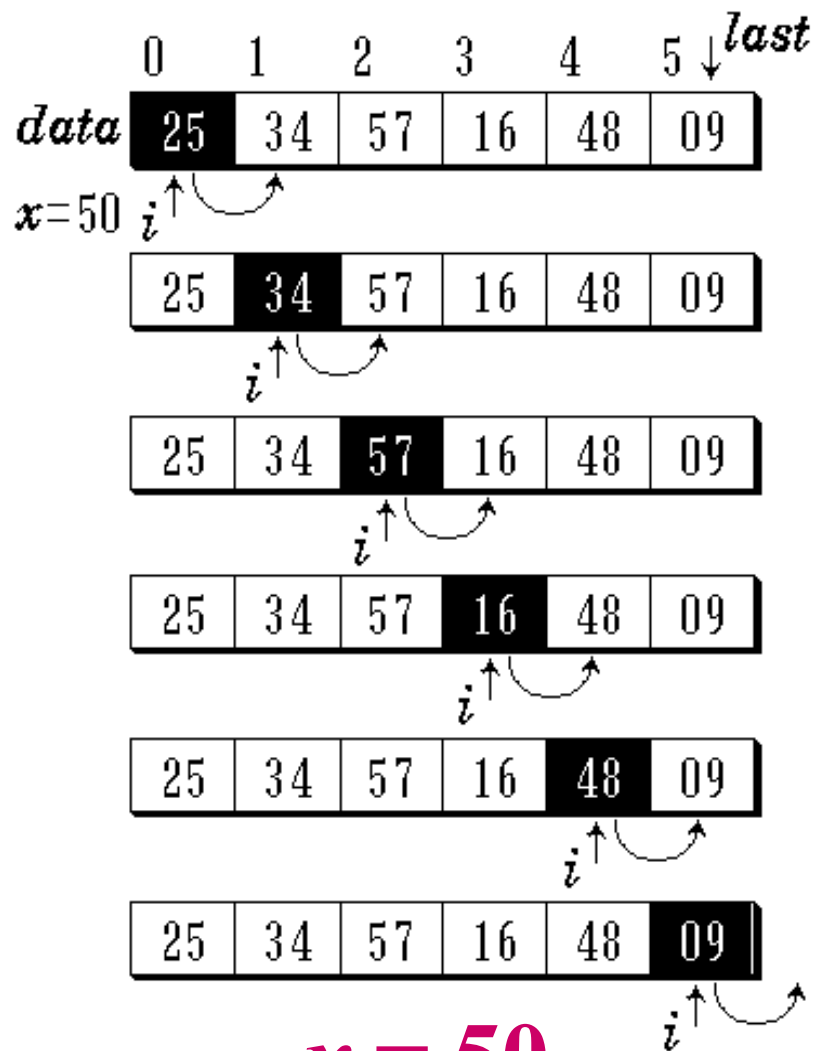
    return 0;                            //搜索失败
};
```



顺序表的搜索 (查找)



$x = 48$



$x = 50$



顺序查找数据的时间代价 (比较次数分析)



ACN(Average Comparing Number)

(假设表的长度为 n , 即 $n = \text{last} + 1$)

搜索成功：表项 i 的查找概率 p_i , 比较次数 c_i

平均比较次数 $ACN = \sum_{i=1}^n p_i * c_i$

若搜索概率 p_i 相等, 则

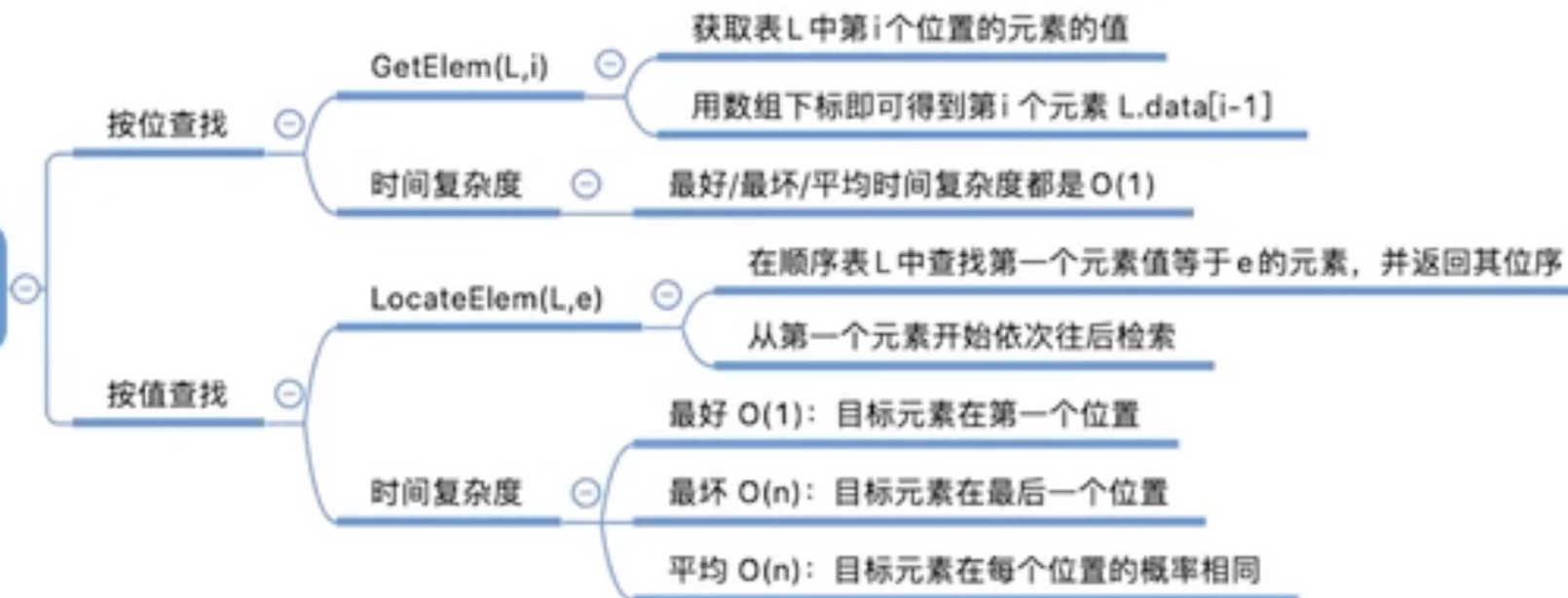
$$ACN = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} (1 + 2 + \dots + n) =$$

$$||||| = \frac{1}{n} * \frac{(1 + n) * n}{2} = \frac{1 + n}{2}$$

搜索不成功：数据比较 n 次

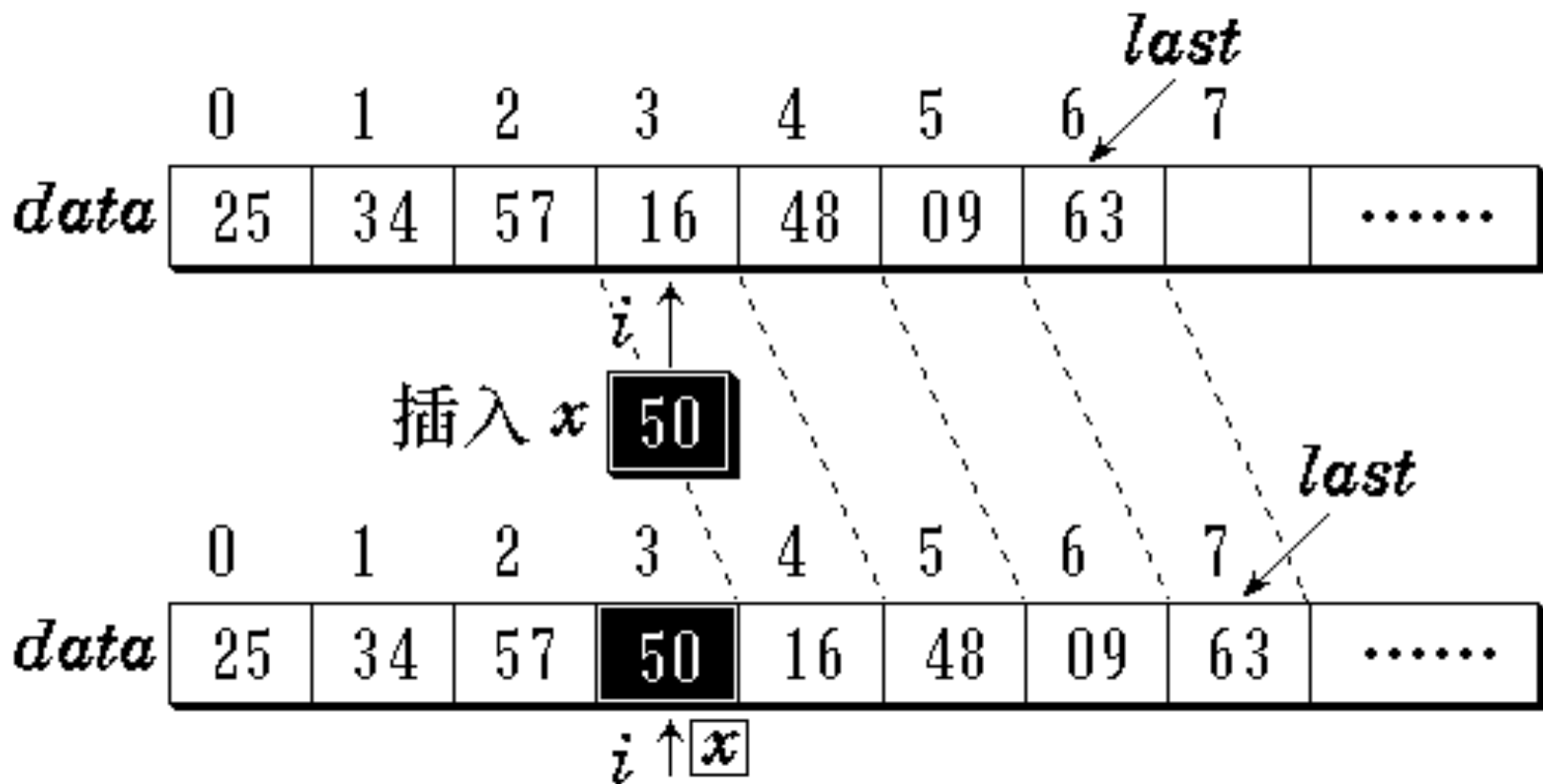


顺序表的基本操作





(4) 顺序表的插入





表项的插入算法



```
template <class E>
bool SeqList<E>::Insert (int i, E x) {
//将新元素x插入到表中第i ( $1 \leq i \leq \text{last}+2$ ) 个表项位置。
    if (last == maxSize-1) return false;           //表满
    if (i < 1 || i > last+2) return false;          //参数i不合理
    for (int j = last; j >= i-1; j--)               //依次后移
        data[j+1] = data[j];
    data[i-1] = x;                                   //插入(第 i 表项在data[i-1]处)
    last++;
    return true;                                     //插入成功
};
```



顺序表插入的时间代价（移动次数）



在表中第 i 个位置插入，从 $\text{data}[i-1]$ 到 $\text{data}[\text{last}]$ 成块后移，移动 $n-1-(i-1)+1 = n-i+1$ 项（假设表的长度为 n ，即 $n = \text{last} + 1$ ）

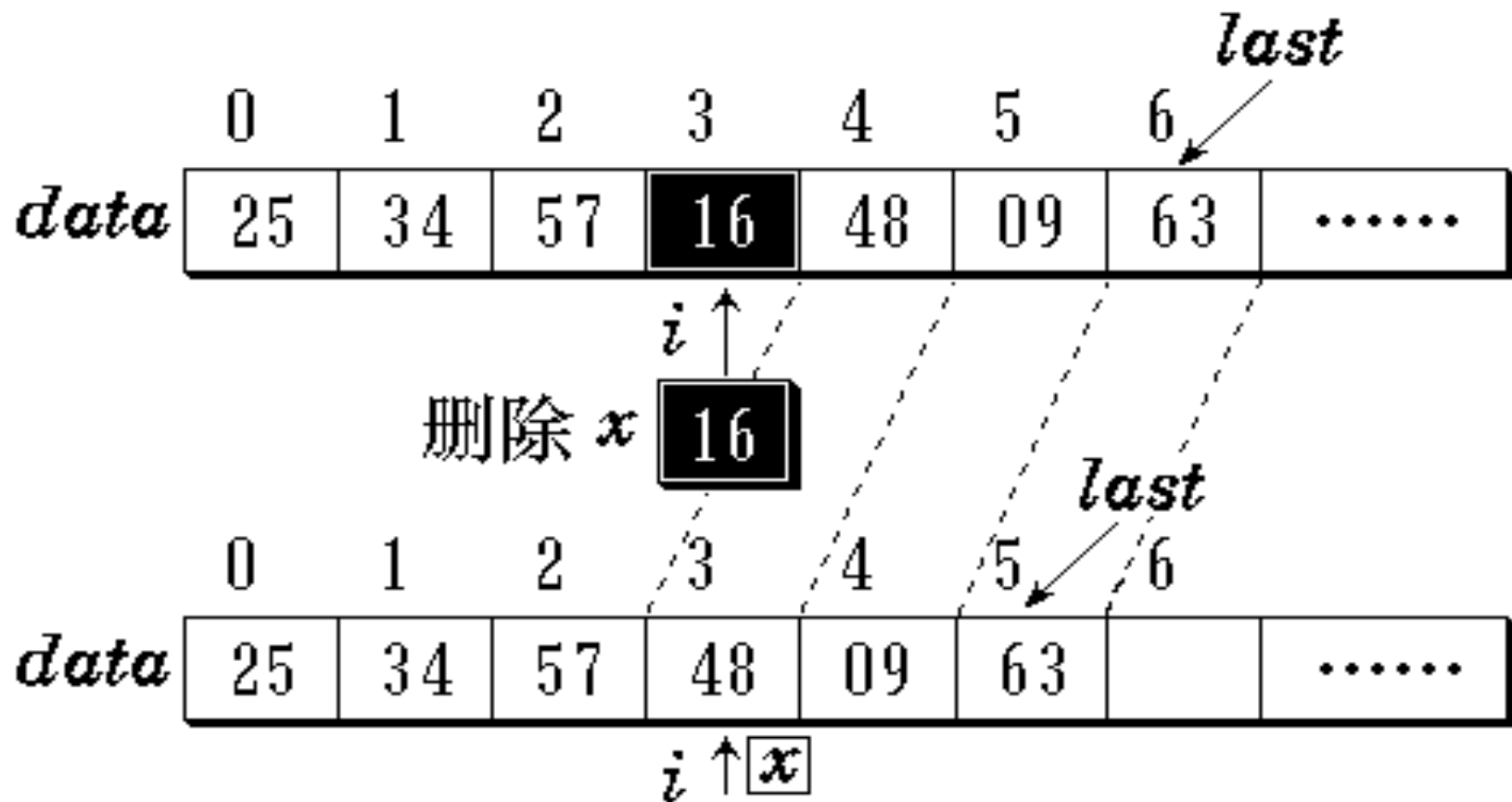
平均数据移动次数 AMN (Average Moving Number) 在各表项插入概率相等时为

$$\begin{aligned} AMN &= \frac{1}{n+1} \sum_{i=1}^{n+1} (n-i+1) = \frac{1}{n+1} (n + \dots + 1 + 0) \\ &= \frac{1}{(n+1)} \frac{n(n+1)}{2} = \frac{n}{2} \end{aligned}$$

在插入时有 $n+1$ 个插入位置，平均移动 $n/2$ 项



(5) 顺序表的表项删除





表项的删除算法



```
template <class E>
bool SeqList<E>::Remove (int i, E& x) {
//从表中删除第 i ( $1 \leq i \leq \text{last}+1$ ) 个表项, 通过引用型
//参数 x 返回被删元素。
    if (last == -1) return false;           //表空
    if (i < 1 || i > last+1) return false; //参数i不合理
    x = data[i-1];
    for (int j = i; j <= last; j++)          //依次前移, 填补
        data[j-1] = data[j];
    last--;
    return true;
};
```



顺序表表项删除的时间代价(移动次数)



删除第 i 个表项，需将第 $i+1$ 项到第 $last+1$ 项全部前移，需前移的项数为 $n-(i+1)+1 = n-i$ (假设表的长度为 n ，即 $n = last + 1$)

平均数据移动次数 AMN (*Average Moving Number*)
在 n 个表项删除概率相等时为

$$AMN = \frac{1}{n} \sum_{i=1}^n (n-i) = \frac{1}{n} \frac{(n-1)n}{2} = \frac{n-1}{2}$$

在删除时有 n 个删除位置，平均移动 $(n-1)/2$ 项



顺序表的应用：用顺序表实现集合的“并”运算



```
void Union ( SeqList<int> & LA,  
            SeqList<int> & LB ) {  
    int n = LA.Length ( );  
    int m = LB.Length ( );  
    int x;  
    for ( int i = 1; i <= m; i++ ) {  
        LB.getData(i, x);           //在LB中取一元素  
        int k = LA.Search (x);      //在LA中搜索它  
        if ( k == 0 )                //若未找到，则插入它  
        { LA.Insert (n, x); n++; }  
    }  
}
```



用顺序表实现集合的“交”运算



```
void Intersection ( SeqList<int> & LA, SeqList<int> & LB ) {  
    int n = LA.Length ( );  
    int m = LB.Length ( ); int i = 1; int x;  
    while ( i <= n ) {  
        LA.getData (i, x);           //在LA中取一元素  
        int k = LB.Search (x);       //在LB中搜索它  
        if ( k == 0 )  
            { LA.Remove (i, x); n--;}  
                                     //未找到, 在LA中删除它  
        else i++;  
    }  
}
```



顺序表

存储结构 ⊖ 逻辑上相邻的数据元素物理上也相邻

实现方式 ⊖

静态分配 ⊖

使用“静态数组”实现

大小一旦确定就无法改变

动态分配 ⊖

使用“动态数组”实现

`L.data = (ElemType *) malloc (sizeof(ElemType) * size);`

顺序表存满时，可再用 malloc 动态拓展顺序表的最大容量

需要将数据元素复制到新的存储区域，并用 free 函数释放原区域

特点 ⊖

随机访问 ⊖

能在 $O(1)$ 时间内找到第 i 个元素

存储密度高

拓展容量不方便

插入、删除数据元素不方便

- 每个节点只存储数据元素
- 即使采用动态分配，时间复杂度也较高
- 需要移动大量元素



线性表

逻辑结构

基本运算/操作

存储/物理结构



顺序表 (顺序存储)

定义 (如何用代码实现)

基本操作的实现

链表 (链式存储)

单链表

定义 (如何用代码实现)

基本操作的实现

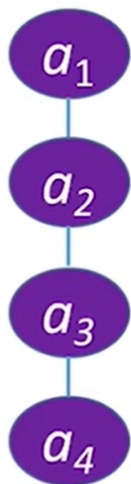
双链表

循环链表

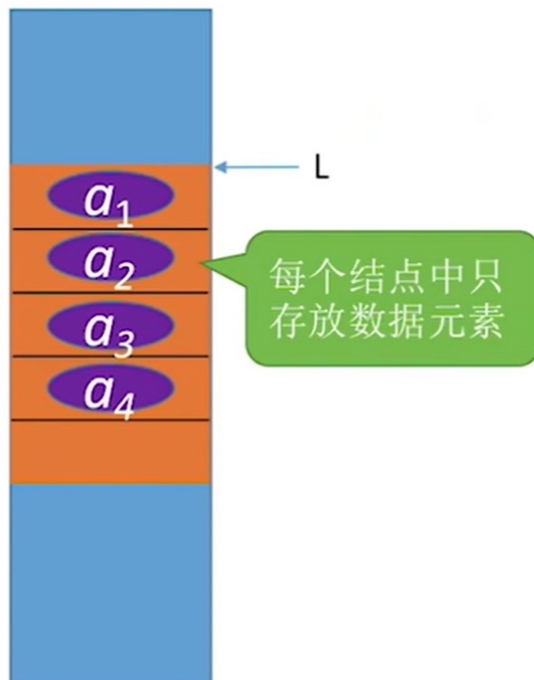
静态链表



逻辑结构：
线性表

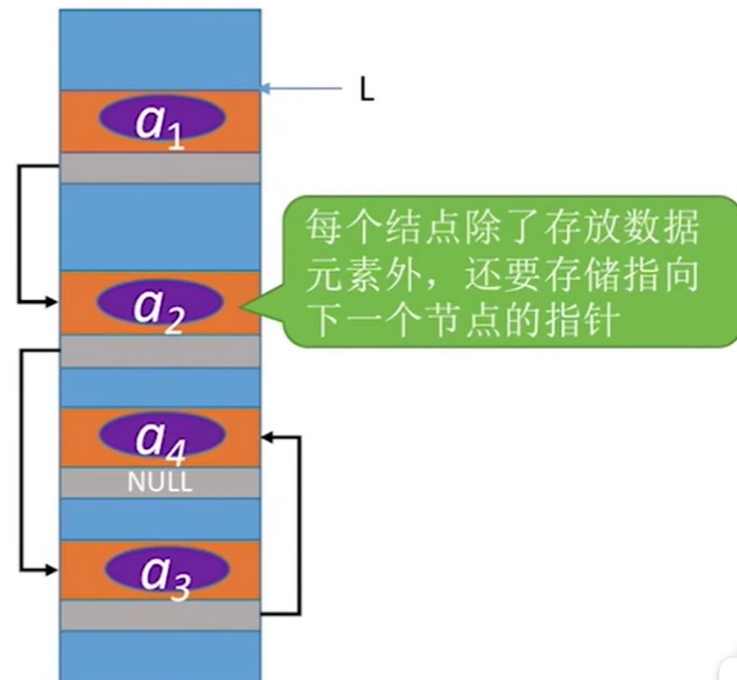


顺序表
(顺序存储)



优点：可随机存取，存储密度高
缺点：要求大片连续空间，改变容量不方便

单链表
(链式存储)



优点：不要求大片连续空间，改变容量方便
缺点：不可随机存取，要耗费一定空间存放指针



§2.3 单链表



连续存储方式（顺序表）

- 特点：存储利用率高，存取速度快
- 缺点：插入、删除等操作时需要移动大量数据

链式存储方式（链表）

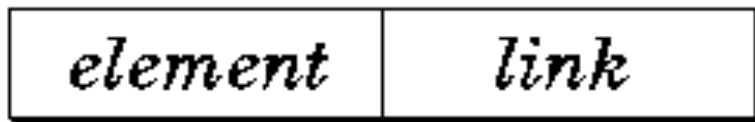
- 特点：适应表的动态增长和删除
- 缺点：需要额外的指针存储空间



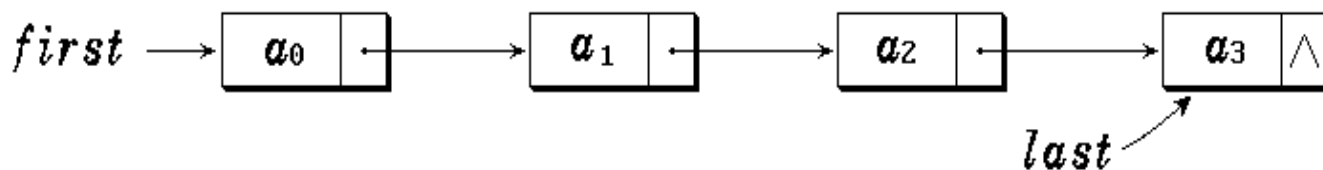
2.3.1 单链表的概念



- 每个元素(表项)由结点(*Node*)构成。



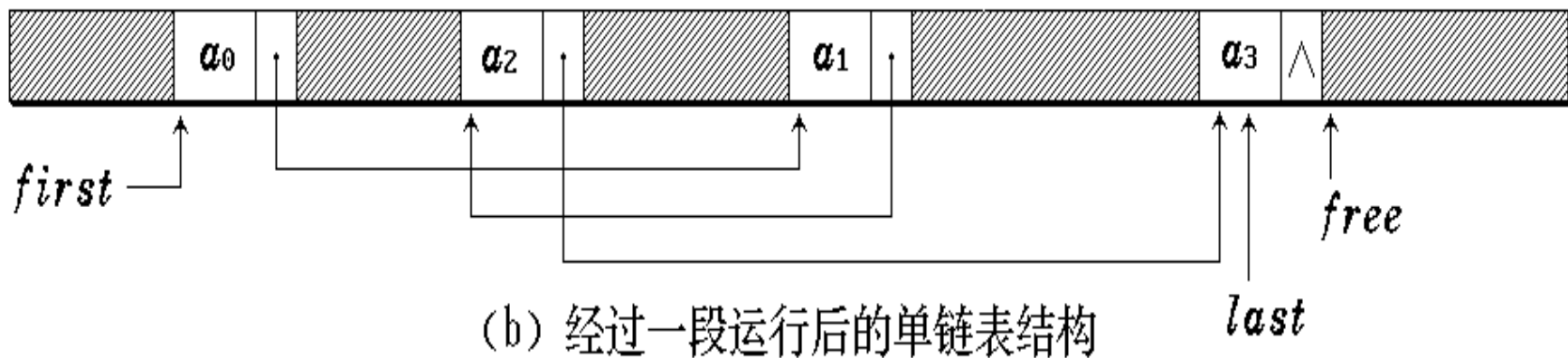
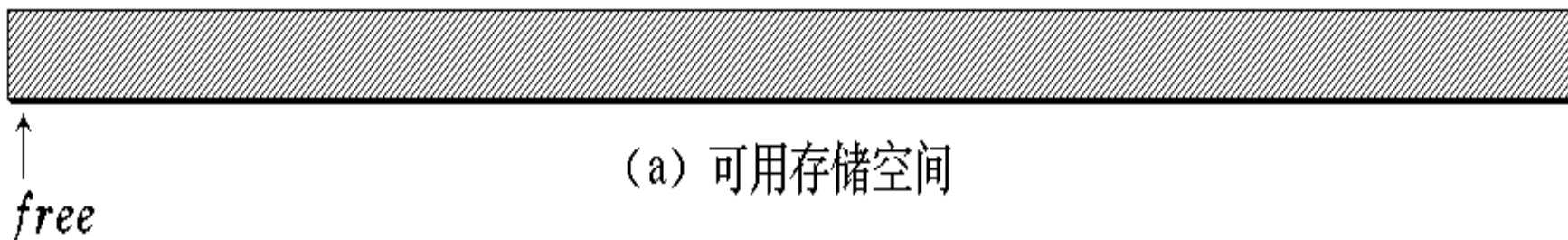
- 线性结构



- 结点可以不连续存储
- 表长度可方便地扩充



单链表的存储映像





2.3.2 单链表的类定义



- **多个类表达一个概念(单链表)**

- 链表结点(*LinkNode*)类
- 链表(*List*)类

- **定义方式**

- 结构方式
- 复合方式
- 嵌套方式
- 继承方式



链表类定义 (结构方式)



```
struct LinkNode {           //链表结点结构
    int data;                 //结点数据, 整型
    LinkNode * link;         //结点指针
};
```

```
class List {                 //链表类
    public:
        ...
    private:
        LinkNode *first ;    //表头指针
};
```



链表类定义（复合方式）



```
class List;
```

//复合方式

```
class LinkNode {  
friend class List;  
private:  
    int data;  
    LinkNode * link;  
};
```

//链表结点类
//链表类为其友元类

//结点数据, **整型**
//结点指针

```
class List {  
private:  
    LinkNode *first ;  
};
```

//链表类

//表头指针



链表类定义（嵌套方式）



```
class List {  
    public:
```

//链表类

```
    ...
```

```
    private:
```

```
        class LinkNode {  
            public:
```

//链表结点类

```
                int data;
```

//结点数据, 整型

```
                LinkNode * link;
```

//结点指针

```
};
```

```
    LinkNode *first ;
```

//表头指针

```
};
```



链表类定义 (继承方式)



```
class LinkNode {           //链表结点类
    protected:
        int data;           //结点数据, 整型
        LinkNode * link;    //结点指针
};
```

```
class List: public class LinkNode { //链表类
    public:
        ...
    private:
        LinkNode *first;           //表头指针
};
```



2.3.3 单链表中的插入与删除操作

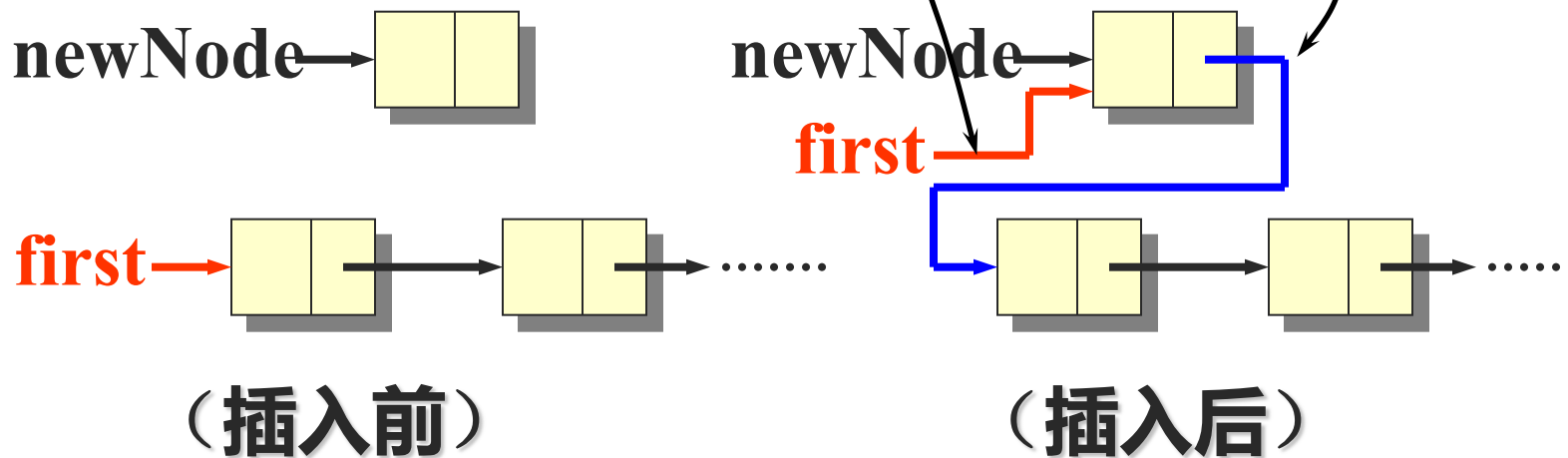


- 插入

- ◆ 第一种情况：在链表最前端插入

`newNode->link = first ;`

`first = newNode;`

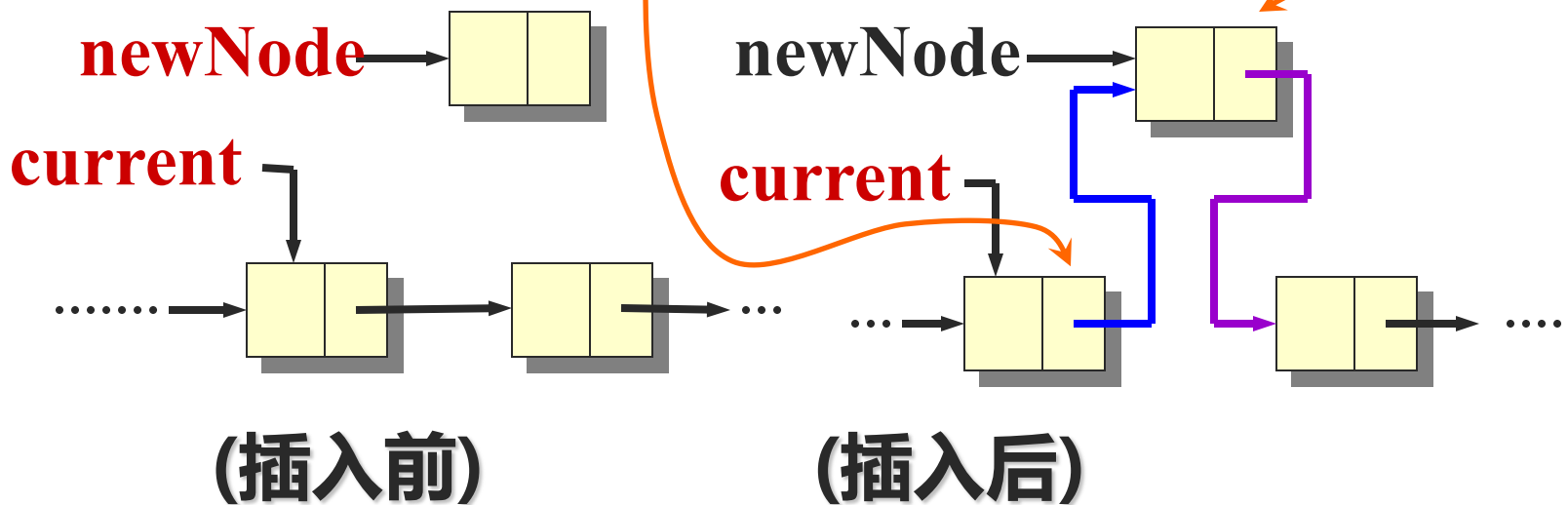




◆ 第二种情况：在链表中间插入

$\text{newNode} \rightarrow \text{link} = \text{current} \rightarrow \text{link};$

$\text{current} \rightarrow \text{link} = \text{newNode};$

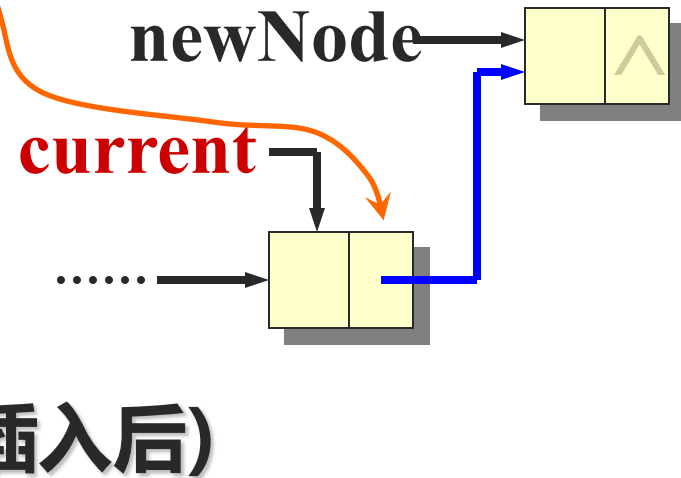
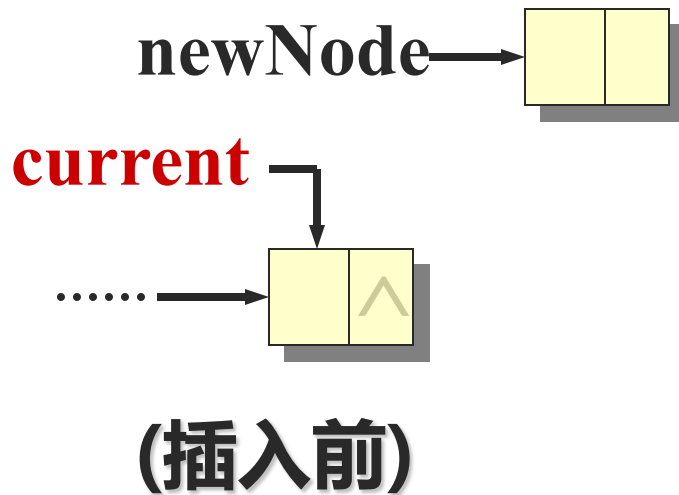




◆ 第三种情况：在链表末尾插入

$\text{newNode} \rightarrow \text{link} = \text{current} \rightarrow \text{link};$

$\text{current} \rightarrow \text{link} = \text{newNode};$





单链表的插入算法



```
bool List::Insert(int i, int x) {  
    //将新元素 x 插入到第 i 个结点之后。i 从1开始,  
    //i = 0 表示插入到首元结点之前。  
    if (first == NULL || i == 0) { //空表或首元结点前  
        LinkNode *newNode = new LinkNode(x);  
        //建立一个新结点  
        newNode->link = first; first = newNode;  
        //新结点成为首元结点  
    }  
    else { //否则, 寻找插入位置  
        LinkNode *current = first; int k = 1;
```

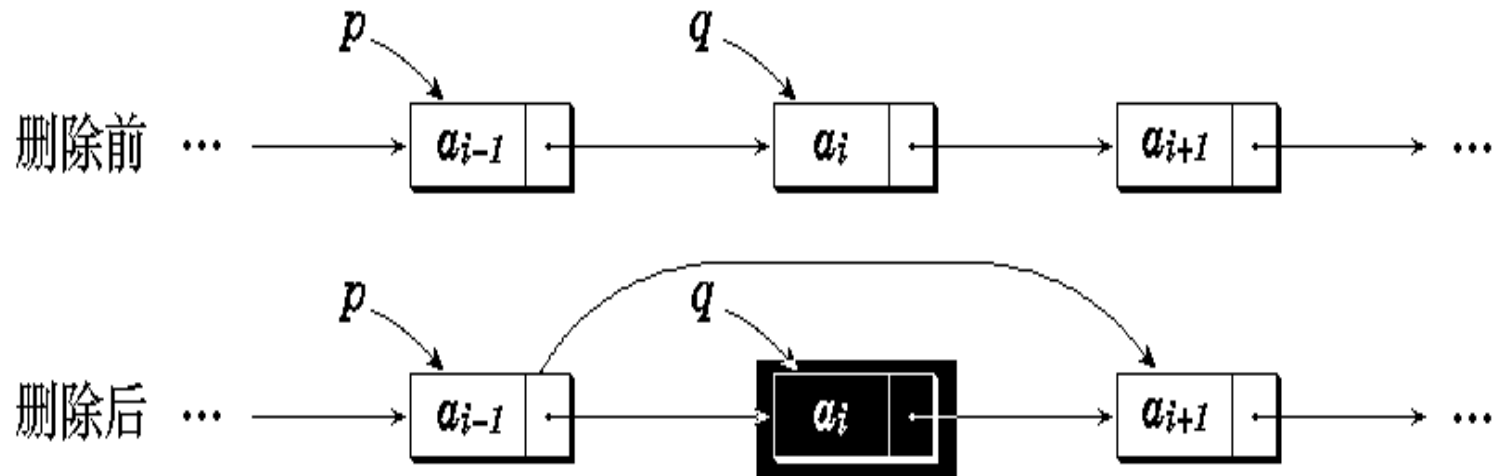


```
while (k < i && current != NULL)    //找第i结点
    { current = current->link; k++; }
if (current == NULL && first != NULL) //链短
    { cerr << “无效的插入位置!\n”; return false; }
else {                                //插入在链表的中间
    LinkNode *newNode = new LinkNode(x);
    newNode->link = current->link;
    current->link = newNode;
    }
}
return true;
};
```



●删除

- 第一种情况: 删除表中第一个元素
- 第二种情况: 删除表中或表尾元素



在单链表中删除含 a_i 的结点



单链表的删除算法



```
bool List::Remove (int i, int& x) {  
    //将链表中的第 i 个元素删去, i 从1开始  
    LinkNode *del;           //暂存删除结点指针  
    if (i <= 1) { del = first; first = first->link; } //删头结点  
    else {  
        LinkNode *current = first; k = 1; //找i-1号结点  
        while (k < i-1 && current != NULL)  
            { current = current->link; k++; }  
        if (current == NULL || current->link == NULL) {  
            cout << “无效的删除位置!\n”; return false;  
        }  
    }  
}
```



```
del = current->link; //删中间/尾结点
current->link = del->link;
}
x = del->data; delete del; //取出被删结点数据
return true;
};
```

- 实现单链表的插入和删除算法，无需移动元素，只需修改结点指针，比顺序表方便。
- 情况复杂，要专门讨论空表和在表头插入的特殊情形。
- 寻找插入或删除位置只能沿着链顺序检测。